

Angular Coding Standard — v2.0

Owner: Jegadhesh · **Approved:** Praveen (CTO)
Applies to: all Angular projects, client and internal
Supersedes: Angular Coding Standard v1.0

Scope and status

This standard applies to **new and changed code**. Existing code is not retrofitted — a rule arriving here does not create a backlog of rewrites. When you touch a file, the code you write in it follows this document.

Where a rule has sample code, that code is the house pattern. Copy it rather than writing your own version.

Angular version policy

Projects run the latest Angular LTS. Upgrades are planned work, scheduled at least once per LTS cycle, not deferred until something forces them. Skipping upgrades for a year turns a routine bump into a migration project and leaves known vulnerabilities in place for that whole period.

This document assumes **Angular 17+**: standalone components, signals, the built-in control flow (`@if` / `@for`), functional interceptors and `takeUntilDestroyed` . A project below 17 follows every rule that applies to its version and schedules the upgrade.

Enforcement tags

Tag	Meaning
[LINTER]	A tool in the repo catches it — ESLint, angular-eslint, Stylelint, <code>tsc</code> .
[CI]	A pipeline step or Git-host setting catches it — Gitleaks, dependency scan, coverage gate, branch protection.
[NEITHER]	A human must judge it in review.
[LINTER + NEITHER]	The linter covers part; review covers the rest.
[CI + NEITHER]	The pipeline covers part; review covers the rest.

Check: notes name the specific tool, or explain why a rule is not automatable. A **backstop** is a narrower rule a tool *can* enforce, used where the real rule cannot be automated — it limits the damage rather than preventing the mistake.

Roughly 45% of this document is `[NEITHER]` . That is the measured reason human review is mandatory on every PR: switch review off and half the standard stops existing.

1. Naming

- `[LINTER]` Files use kebab-case with a type suffix: `user-profile.component.ts` , `auth.service.ts` , `role.guard.ts` , `date-ago.pipe.ts` . *Reason:* the file name alone tells a reviewer what the file is without opening it. **Check:** `eslint-plugin-check-file` .
- `[LINTER]` Classes PascalCase, members and functions camelCase, constants UPPER_SNAKE_CASE. **Check:** `@typescript-eslint/naming-convention` .
- `[LINTER]` Component selectors are prefixed (`app-` , or a feature prefix in shared libraries). *Reason:* makes our elements distinguishable from third-party ones in DevTools. **Check:** `@angular-eslint/component-selector` , `@angular-eslint/directive-selector` .
- `[LINTER + NEITHER]` Observables end in `$` (`users$`). Signals do not. *Reason:* at a call site you must know whether the value needs subscribing. **Check:** Linter — `rxjs-x/finnish` (type-aware). Human — no rule stops a signal being named with `$` .
- `[LINTER + NEITHER]` Booleans read as assertions: `isLoading` , `hasAccess` , `canSubmit` . Handlers are `onX` for template events, `handleX` for internal logic. No `data` , `info` , `temp` , `flag` , `obj` as a name — if that is the best name available, the variable is doing too much. **Check:** Linter — `@typescript-eslint/naming-convention` , `id-denylist` . Human — `onX` / `handleX` convention.

1.1 Wire format across stacks

[NEITHER] JSON payloads are camelCase on the wire, in both directions.

Our Python services use `snake_case` internally and our Node services use `camelCase`. Both are correct in their own language. What matters is the contract between them, and it is camelCase.

Python converts at the boundary with Pydantic field aliases. Angular never converts: what arrives from the API is already in the shape our interfaces declare.

If an endpoint returns `snake_case`, that is a backend defect — raise it rather than mapping around it in the frontend. A per-feature mapping layer is how one inconsistent endpoint becomes twelve.

2. Project structure

Feature-first, not type-first. *Reason*: a feature is what gets built, reviewed and deleted as a unit; grouping by type spreads one change across five folders.

```
src/app/
  core/           # singletons: interceptors, guards, auth, config, logging
  shared/         # reusable dumb components, pipes, directives
  features/
    user/
      pages/      # routed containers
      components/ # presentational, feature-local
      services/
      models/
      user.routes.ts
  layouts/
  app.config.ts
```

- **[NEITHER]** **Feature-first on every new project. No exceptions.** The layout costs nothing on a small project — the same files in differently named folders — and a project that stays small is rare enough that planning for it is not worth the opt-out. Existing projects on another layout are not migrated; there is no return on that work. (*Avoiding premature abstraction is a real concern and is handled by the “earn it” rules in section 10, not by changing the folder structure.*)
- **[NEITHER]** Anything used by two or more features moves to `shared/`. Used once, it stays inside the feature.
- **[NEITHER]** Every routed feature is lazy-loaded via `loadChildren/` `loadComponent`. *Reason*: initial bundle size is our most visible performance number. **Check**: no standard lint rule verifies this; `angular.json` bundle budgets fail the build on the effect, not the rule.
- **[LINTER]** No cross-feature imports. Features talk through `core/` or `shared/`, never directly to another feature’s internals. **Check**: `eslint-plugin-boundaries` or `import/no-restricted-paths`.

3. Error handling & validation

- **[NEITHER]** HTTP errors are handled in one central interceptor plus a global `ErrorHandler`. Components do not write their own generic error toasts. **Check**: the interceptor exists once per project; whether a component adds its own toast is a judgement.
- **[NEITHER]** Components handle only errors they can act on — field-level validation, retry, empty state. Everything else propagates.
- **[NEITHER]** Users never see a raw error object, stack trace or backend message. Map to a human message; keep the technical detail in the log.
- **[LINTER + NEITHER]** Forms use typed reactive forms. Validation rules live in the form definition, not in `(click)` handlers. **Check**: Linter — `no-restricted-imports` banning `UntypedFormGroup`, `UntypedFormControl`, `UntypedFormBuilder`. Human — whether validation lives in the form definition.
- **[NEITHER]** Every async view has four explicit states: **loading, empty, error, success**. A screen that only handles the happy path is not done.

3.1 The error contract

Our backends return a consistent error body. Angular depends on that shape and maps it once.

```
// src/app/core/http/api-error.ts

/** The body every one of our APIs returns on failure. */
export interface ApiErrorBody {
```

```

code: string;           // machine-readable: 'LOAN_ALREADY_SETTLED'
message: string;        // developer-facing; NEVER shown to a user
details?: Record<string, string[]>; // field name -> validation messages
}

export class ApiError extends Error {
  constructor(
    readonly code: string,
    readonly status: number,
    readonly correlationId: string | null,
    readonly details?: Record<string, string[]>,
    developerMessage?: string,
  ) {
    super(developerMessage ?? code);
    this.name = 'ApiError';
  }

  /** Field errors for a reactive form, or null. */
  get fieldErrors(): Record<string, string[]> | null {
    return this.details ?? null;
  }
}

```

Branch on **code**, never on **message**. Message text changes; codes are a contract.

3.2 The error interceptor

```

// src/app/core/http/error.interceptor.ts
import { HttpResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const log = inject(LoggerService);

  return next(req).pipe(
    catchError((err: HttpResponse) => {
      const correlationId = err.headers?.get('X-Correlation-Id') ?? null;

      // status 0 = network failure, CORS, or the request was blocked.
      if (err.status === 0) {
        log.error('Network request failed', { url: req.url, correlationId });
        return throwError(() => new ApiError('NETWORK_UNAVAILABLE', 0, correlationId));
      }

      const body = err.error as ApiErrorBody | null;
      const apiError = new ApiError(
        body?.code ?? 'UNKNOWN_ERROR',
        err.status,
        correlationId,
        body?.details,
        body?.message,
      );

      // 4xx is usually the user's problem and is handled locally.
      // 5xx is ours – it goes to Sentry.
      if (err.status >= 500) {
        log.error('Server error', { url: req.url, status: err.status, code: apiError.code, correlationId });
      }

      return throwError(() => apiError);
    }),
  );
};

```

3.3 Turning an error into something a user can read

```

// src/app/core/http/error-messages.ts

const MESSAGES: Record<string, string> = {
  NETWORK_UNAVAILABLE: 'We could not reach the server. Check your connection and try again.',
  LOAN_ALREADY_SETTLED: 'This loan has already been settled.',
  INSUFFICIENT_BALANCE: 'Your balance is too low to complete this.',
};

const FALLBACK = 'Something went wrong. Please try again.';

```

```
export function userMessage(error: unknown): string {
  return error instanceof ApiError ? MESSAGES[error.code] ?? FALLBACK : FALLBACK;
}
```

Unmapped codes fall back deliberately. **Never show `error.message` to a user** — it is written for developers and may carry internal detail.

3.4 The canonical async view

Every screen that loads data looks like this. Four states, no exceptions.

```
// src/app/features/loans/pages/loan-list.page.ts
import { Component, ChangeDetectionStrategy, inject, signal } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

type ViewState<T> =
  | { status: 'loading' }
  | { status: 'empty' }
  | { status: 'error'; message: string; correlationId: string | null }
  | { status: 'ready'; data: T };

@Component({
  selector: 'app-loan-list',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  templateUrl: './loan-list.page.html',
})
export class LoanListPage {
  private readonly loans = inject(LoanService);
  readonly state = signal<ViewState<Loan[]>>({ status: 'loading' });

  constructor() {
    this.loans.list()
      .pipe(takeUntilDestroyed())
      .subscribe({
        next: data => this.state.set(
          data.length ? { status: 'ready', data } : { status: 'empty' },
        ),
        error: (e: ApiError) => this.state.set({
          status: 'error',
          message: userMessage(e),
          correlationId: e.correlationId,
        }),
      });
  }
}
```

```
<!-- loan-list.page.html -->
@switch (state().status) {
  @case ('loading') { <app-spinner /> }
  @case ('empty') { <app-empty-state message="You have no loans yet." /> }
  @case ('error') {
    <app-error-state [message]="state().message" />
    @if (state().correlationId) {
      <small>Reference: <code>{{ state().correlationId }}</code></small>
    }
  }
  @case ('ready') {
    @for (loan of state().data; track loan.id) {
      <app-loan-card [loan]="loan" />
    }
  }
}
```

Show the correlation ID on the error state. It costs nothing and turns a support ticket into a log lookup.

4. Logging & observability

- [LINTER] No `console.log` in committed code. Use `LoggerService`. Reason: `console.log` cannot be switched off, filtered or shipped anywhere. Check: `no-console`, with `core/logging/**` exempted.
- [CI] `debug` is stripped in production builds. Check: a `LoggerService` unit test asserting `debug()` is a no-op under production config.

- [NEITHER] **Never logged, in any environment:** tokens, passwords, OTPs, API keys, wallet private keys, seed phrases, `Authorization` headers, card or bank data, KYC documents, full API responses containing personal data. **Check:** backstop — the redaction list in `LoggerService` enforces this in code (4.2).
- [NEITHER] Production minimum: unhandled errors and failed HTTP calls reach Sentry with route, user id (never name or email) and correlation id. **Check:** verified once per project at setup.

4.1 Log levels

The same four meanings across every stack in the company.

Level	Meaning
error	A human needs to look at this.
warn	Degraded but handled.
info	A state change worth auditing.
debug	Off in production.

One `info` line per HTTP call means nobody reads `info` at all.

4.2 Redaction is enforced in code

A denylist in a document is a hope. A denylist in a function is a control.

```
// src/app/core/logging/redact.ts

const DENY = [
  /^authorization$/i, /password$/i, /passwd$/i, /secret$/i, /token$/i,
  /^otp$/i, /^pin$/i, /apikey$/i, /api_key$/i, /privatekey$/i, /private_key$/i,
  /mnemonic$/i, /seed$/i, /^cvv$/i, /cardnumber$/i, /card_number$/i,
  /^aadhaar$/i, /^pan$/i, /^ssn$/i, /^email$/i, /^phone$/i,
];

const REDACTED = '[REDACTED]';
const MAX_DEPTH = 6;

export function redact(value: unknown, depth = 0): unknown {
  if (depth > MAX_DEPTH) return '[TRUNCATED]';
  if (value === null || typeof value !== 'object') return value;
  if (Array.isArray(value)) return value.map(v => redact(v, depth + 1));

  const out: Record<string, unknown> = {};
  for (const [key, val] of Object.entries(value as Record<string, unknown>)) {
    out[key] = DENY.some(rx => rx.test(key)) ? REDACTED : redact(val, depth + 1);
  }
  return out;
}
```

If something sensitive does reach a log, treat it as a leaked credential and rotate it. The log has already been shipped to Sentry and retained. Removing the log line changes nothing.

4.3 LoggerService

```
// src/app/core/logging/logger.service.ts
import { Injectable } from '@angular/core';
import * as Sentry from '@sentry/angular';

type Level = 'debug' | 'info' | 'warn' | 'error';

@Injectable({ providedIn: 'root' })
export class LoggerService {
  private get isProd(): boolean {
    return currentConfig().environment === 'production';
  }

  debug(msg: string, ctx?: unknown): void {
```

```

    if (!this.isProd) this.write('debug', msg, ctx);
  }

  info(msg: string, ctx?: unknown): void {
    this.write('info', msg, ctx);
    Sentry.addBreadcrumb({ level: 'info', message: msg, data: redact(ctx) as never });
  }

  warn(msg: string, ctx?: unknown): void {
    this.write('warn', msg, ctx);
    Sentry.addBreadcrumb({ level: 'warning', message: msg, data: redact(ctx) as never });
  }

  /** error = a human needs to look at this. Not "an exception happened". */
  error(msg: string, ctx?: unknown): void {
    this.write('error', msg, ctx);
    Sentry.captureMessage(msg, { level: 'error', extra: redact(ctx) as never });
  }

  private write(level: Level, msg: string, ctx?: unknown): void {
    if (this.isProd && level === 'debug') return;
    const safe = ctx === undefined ? '' : JSON.stringify(redact(ctx));
    console[level === 'debug' ? 'log' : level](`[${level}] ${msg}`, safe);
  }
}

```

4.4 Sentry

Sentry initialises **after** config loads, because the DSN comes from `config.json`.

```

// src/app/core/observability/sentry.setup.ts
import * as Sentry from '@sentry/angular';

export function initSentry(config: AppConfig): void {
  if (!config.sentryDsn) return; // local development

  Sentry.init({
    dsn: config.sentryDsn,
    environment: config.environment,
    release: config.release,
    tracesSampleRate: config.environment === 'production' ? 0.1 : 1.0,
    sendDefaultPii: false,

    // Second line of defence behind redact(). Anything that slips through
    // LoggerService is scrubbed here before it leaves the browser.
    beforeSend(event) {
      if (event.request?.headers) {
        event.request.headers = redact(event.request.headers) as Record<string, string>;
      }
      if (event.extra) event.extra = redact(event.extra) as Record<string, unknown>;
      if (event.contexts) event.contexts = redact(event.contexts) as never;

      // Query strings carry tokens more often than anyone expects.
      if (event.request?.url) event.request.url = event.request.url.split('?')[0];

      return event;
    },

    beforeSendBreadcrumb(crumb) {
      if (crumb.category === 'console') return null; // console is already logged
      if (crumb.data) crumb.data = redact(crumb.data) as Record<string, unknown>;
      return crumb;
    },
  });
}

/** User id only. Never name, email or phone. */
export function identifyUser(userId: string): void {
  Sentry.setUser({ id: userId });
}

export function clearUser(): void {
  Sentry.setUser(null);
}

```

Wire the global handler in `app.config.ts`:

```

{ provide: ErrorHandler, useValue: Sentry.createErrorHandler({ showDialog: false }) },

```

`sendDefaultPii: false` is not optional. Call `clearUser()` on logout.

4.5 Correlation IDs

```
// src/app/core/http/correlation-id.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';
import * as Sentry from '@sentry/angular';

export const correlationIdInterceptor: HttpInterceptorFn = (req, next) => {
  const id = crypto.randomUUID();
  Sentry.setTag('correlation_id', id);
  return next(req.clone({ setHeaders: { 'X-Correlation-Id': id } }));
};
```

Our backends log this header and return it. One ID follows a request from the browser through every service — that is the difference between investigating a failure and guessing at it.

5. Testing expectations

The developer proves the code works before it reaches QA. QA validates quality and risk; QA is not the first person to discover whether a feature runs.

- **[NEITHER]** **Must** have unit tests: services with logic, guards, interceptors, pipes, pure functions, form validators. Cheap to test, expensive to break.
- **[NEITHER]** Presentational components need a test only when they hold logic — conditional rendering, output emission. Snapshot-only tests are not a test.
- **[CI]** New and changed lines meet the project's coverage threshold; overall coverage must not drop in a PR. *Reason:* a percentage target on legacy code is meaningless; a ratchet on new code is not. **Check:** Karma/Jest `coverageThreshold` plus a diff-coverage step (SonarQube new-code gate or Codecov patch status).
- **[NEITHER]** Every fixed bug ships with a test that fails without the fix. **This is our only real regression suite.**
- **[LINTER]** No `fdescribe / fit`; skipped tests need a linked ticket in a comment. **Check:** `eslint-plugin-jasmine` `no-focused-tests`, `no-disabled-tests`; `eslint-comments/require-description` forces the reason.
- **[NEITHER]** Tests never run against live third-party APIs, mainnet, or production data. A test that depends on an external system fails randomly, then gets ignored, then gets deleted.
- **[CI]** Existing tests pass before merge. A developer does not disable, skip or delete a failing test to get a PR through. If a test is wrong, it is fixed and the reason goes in the PR.

6. PR & merge rules

- **[CI]** **One approval minimum, from someone other than the author, on every PR in every project.** Two approvals when the change touches `core/`, `auth`, interceptors or routing. **Check:** branch protection plus CODEOWNERS (GitHub) or path-based approval rules (GitLab).
- **[CI]** Blocks merge: failing CI, lint or build errors, self-approval, protected-branch bypass, unresolved blocking comments, dropped coverage. **Check:** pipeline steps and branch-protection settings.
- **[NEITHER]** One PR addresses one ticket. Unrelated refactors, formatting sweeps and dependency bumps go in their own PR. *Reason:* a reviewer who cannot tell the fix from the noise approves the noise.
- **[NEITHER]** Target **400 changed lines**, excluding lock files, generated code and pure renames. Larger PRs carry a note saying why and how to review them. *Reason:* past that point a reviewer skims instead of reads, and approval stops meaning anything.
- **[LINTER]** Branches: `feature/<ticket>-<slug>`, `fix/<ticket>-<slug>`. Commits describe the change in plain English.
- **[CI + NEITHER]** Required PR description fields: what changed · why · how it was tested · security/architecture impact · breaking changes · related ticket · deployment requirements. **Check:** CI — PR template plus a check that fields are filled. Human — whether the answers mean anything. *"Tested locally" is not an answer.*
- **[CI]** Gitleaks and a dependency scan run on every PR. A detected secret blocks the merge **and triggers rotation of that credential** — once committed it is in the history and must be treated as leaked.

On the review requirement. About 45% of the rules in this document are [NEITHER] — no tool enforces them. On a project where review is switched off, nearly half this standard does not exist, including most of section 8. That is why the rule is per-company and not per-project. Exceptions are granted by the CTO for a named person, not chosen per project.

7. Review checklist

1. [NEITHER] Does the change match the ticket, and nothing more?
2. [LINTER] Any subscription without `takeUntilDestroyed` or the async pipe? **Check:** `rxjs-angular/prefer-takeuntil`.
3. [LINTER] Any `any`, `as`, or `!` that could be a real type? **Check:** `@typescript-eslint/no-explicit-any`, `no-non-null-assertion`, `consistent-type-assertions`.
4. [LINTER + NEITHER] Function calls or heavy expressions in the template? **Check:** `@angular-eslint/template/no-call-expression`, configured to permit signal reads (see 10).
5. [NEITHER] `@for` blocks tracking a stable id rather than `$index`?
6. [NEITHER] Loading, empty, error and success states all handled?
7. [NEITHER] Business logic in a component that belongs in a service?
8. [LINTER + NEITHER] Hardcoded strings, URLs, IDs or magic numbers? **Check:** `@typescript-eslint/no-magic-numbers`; URLs and UI strings judged by a human.
9. [NEITHER] New dependency, and is it justified in the description?
10. [NEITHER] Anything sensitive reaching a log, a URL, or Sentry?
11. [NEITHER] Tests added for the logic changed and for the bug being fixed?
12. [LINTER] Commented-out code, leftover TODOs, unused imports? **Check:** `sonarjs/no-commented-code`, `no-warning-comments`, `@typescript-eslint/no-unused-vars`.
13. [NEITHER] **Money handled per section 12** — no `number` arithmetic on an amount?
14. [NEITHER] Can it be used with a keyboard, and does every control have an accessible name?
15. [LINTER] Any raw colour, spacing or font-size value instead of a token? **Check:** `stylelint-declaration-strict-value`.
16. [NEITHER] Is a presentational component injecting a service, or setting its own outer margin?
17. [NEITHER] Hand-written CSS outside the four permitted places in 13.2?

8. Security & secrets

8.1 Nothing secret ships to the browser

[CI] No API key, client secret, signing key, private key or credential appears anywhere in the repo — including `environment.ts`, `config.json`, constants and comments.

Reason: everything in the bundle is public. Minification is not obfuscation and obfuscation is not encryption.

Prove it once:

```
npm run build
grep -rInE "sk_live|sk_test|AKIA|AIza|xox[baprs]-|----BEGIN" dist/ | head
```

If that prints anything, the key is already public and must be **rotated**, not deleted.

The pattern: when a feature needs a third-party service that requires a key, the browser calls our API and our API calls the third party.

```
x Browser —key→ ThirdParty
✓ Browser —————→ Our API —key from AWS Secrets Manager→ ThirdParty
```

There is no exception. If a vendor's documentation shows a browser-side key, that key is either publishable by design (a Stripe *publishable* key, a Mapbox public token) or the documentation is showing a prototype. **If you are unsure which you are holding, you are holding a secret.**

Check: Gitleaks on every PR, blocking.

8.2 Configuration is loaded at runtime

[NEITHER] One build artifact runs in every environment. Environment-specific values come from `/config.json`, fetched before the app boots.

Reason: baking config into the build means staging and production run different artifacts, so what you tested is not what you shipped. It also creates the habit of putting values in `environment.prod.ts` — which is where someone eventually puts a key.

```
// src/app/core/config/app-config.ts
import { InjectionToken } from '@angular/core';

/** Everything here is PUBLIC. It is served as a plain file. */
export interface AppConfig {
  apiBaseUrl: string;
  environment: 'local' | 'dev' | 'staging' | 'production';
  release: string;
  sentryDsn?: string;          // a DSN is public by design
  chainId?: number;
  contracts?: Record<string, `0x${string}`>;
  featureFlags: Record<string, boolean>;
}

export const APP_CONFIG = new InjectionToken<AppConfig>('APP_CONFIG');
```

```
// src/app/core/config/config.loader.ts
let loaded: AppConfig | null = null;

export async function loadAppConfig(): Promise<void> {
  const http = inject(HttpClient);
  loaded = await firstValueFrom(
    http.get<AppConfig>(`/config.json?v=${Date.now()}`), // defeat a stale CDN copy
  );
  initSentry(loaded);
}

export function currentConfig(): AppConfig {
  if (!loaded) throw new Error('Config read before APP_INITIALIZER completed');
  return loaded;
}
```

```
// src/app/app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withInterceptors([
      correlationIdInterceptor, // first: present even on requests that fail auth
      authInterceptor,
      errorInterceptor,
    ])),
    { provide: APP_INITIALIZER, useValue: loadAppConfig, multi: true },
    { provide: APP_CONFIG, useFactory: currentConfig },
    { provide: ErrorHandler, useValue: Sentry.createErrorHandler({ showDialog: false }) },
  ],
};
```

Angular 19+: replace the `APP_INITIALIZER` provider with `provideAppInitializer(loadAppConfig)`.

AWS: we host on S3 behind CloudFront. The deploy writes `config.json` per environment. Set `Cache-Control: no-cache` on that one object; everything else is content-hashed and cached hard.

8.3 Session tokens are managed by the backend

[NEITHER] The session is an `httpOnly`, `Secure`, `SameSite` cookie set by the backend. JavaScript never reads it.

Where a backend cannot set a cookie, the access token is held in a **service field in memory** and nowhere else. Never `localStorage`, never `sessionStorage`, never a JS-readable cookie.

Reason: `localStorage` is readable by any script on the page, including one that arrives through a compromised npm package. An `httpOnly` cookie is not readable by script at all.

```
// src/app/core/auth/token.store.ts
import { Injectable, signal } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class TokenStore {
  /** In memory only. Never persisted. Never logged. */
  private accessToken: string | null = null;
  readonly isAuthenticated = signal(false);

  get(): string | null { return this.accessToken; }
  set(token: string): void { this.accessToken = token; this.isAuthenticated.set(true); }
  clear(): void { this.accessToken = null; this.isAuthenticated.set(false); }
}
```

Never, in any form, including “just for testing”:

```
localStorage.setItem('token', accessToken); // ✗
sessionStorage.setItem('auth', JSON.stringify(x)); // ✗
document.cookie = `token=${accessToken}`; // ✗ readable by script
```

Check: backstop — `no-restricted-properties` on `localStorage` / `sessionStorage` outside one storage service.

```
'no-restricted-properties': ['error',
  { object: 'localStorage', property: 'setItem', message: 'Tokens and PII must not be persisted. See standard 8.3.' },
  { object: 'sessionStorage', property: 'setItem', message: 'Tokens and PII must not be persisted. See standard 8.3.' },
],
```

8.4 One refresh at a time

[NEITHER] When several requests fail with 401 simultaneously, **exactly one** refresh call is made. Every waiting request joins it and retries once.

Reason: the naive interceptor fires one refresh per failed request. Refresh tokens rotate on use, so the first succeeds and the rest present a consumed token — the backend treats that as replay, revokes the session, and the user is logged out at random. It is intermittent, never reproduces locally, and is usually blamed on the backend.

```
// src/app/core/auth/auth.service.ts
export const SKIP_AUTH = new HttpContextToken<boolean>(() => false);
export const skipAuth = () => new HttpContext().set(SKIP_AUTH, true);

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly http = inject(HttpClient);
  private readonly store = inject(TokenStore);
  private readonly router = inject(Router);

  /** The one in-flight refresh, or null. */
  private refresh$: Observable<string> | null = null;

  refreshAccessToken(): Observable<string> {
    if (this.refresh$) return this.refresh$; // join the running refresh

    this.refresh$ = this.http
      .post<{ accessToken: string }>(
        `${currentConfig().apiBaseUrl}/auth/refresh`,
        {},
        { withCredentials: true, context: skipAuth() }, // or it 401s and recurses
      )
      .pipe(
        map(res => res.accessToken),
        tap(token => this.store.set(token)),
        catchError(err => {
          this.store.clear();
          clearUser();
          this.router.navigate(['/login'], { queryParams: { returnUrl: this.router.url } });
          return throwError(() => err);
        }),
        finalize(() => { this.refresh$ = null; }),
        shareReplay({ bufferSize: 1, refCount: false }),
      );

    return this.refresh$;
  }
}
```

```
// src/app/core/auth/auth.interceptor.ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const store = inject(TokenStore);
  const auth = inject(AuthService);

  if (req.context.get(SKIP_AUTH)) return next(req);

  const withToken = (token: string | null) =>
    token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;

  return next(withToken(store.get())).pipe(
    catchError((err: HttpResponse) => {
      if (err.status !== 401) return throwError(() => err);
      return auth.refreshAccessToken().pipe(
        switchMap(token => next(withToken(token))),
      );
    }),
  );
};
```

Why this retries exactly once: the retry is issued inside `switchMap`, downstream of the `catchError`. A second 401 propagates out without re-entering the handler. The `SKIP_AUTH` context on the refresh call stops the other possible loop.

Check: review — any `Authorization` header set anywhere other than this interceptor is a defect.

8.5 Route guards are UX, not security

[NEITHER] Every guard has a matching server-side authorisation check.

Reason: a guard runs in code the user controls. The concrete failure is **IDOR** — a valid token for user A, with user B's id in the URL, returning user B's data. The backend prevents this by scoping the query to the caller. The frontend cannot prevent it and must not be relied on to.

```
export const roleGuard = (...allowed: string[]): CanActivateFn => () => {
  const session = inject(SessionStore);
  const router = inject(Router);
  if (allowed.some(role => session.roles().includes(role))) return true;
  // UX only. The API enforces this independently and authoritatively.
  return router.createUrlTree(['/forbidden']);
};
```

Check: review — for every guard added or changed, name the enforcing API endpoint in the PR description.

8.6 User input is never rendered as HTML

[LINTER + NEITHER] No `innerHTML` with anything a user can influence. `bypassSecurityTrust*` requires an approval comment and a second reviewer.

```
// APPROVED BYPASS
// Source: /api/v1/cms/articles – server-side sanitised with DOMPurify before storage.
// Reviewed: <lead name>, <date>. See standard 8.6.
readonly body = computed(() =>
  this.sanitizer.bypassSecurityTrustHtml(this.article().sanitisedHtml)
);
```

Check: Linter — `no-restricted-syntax` on `bypassSecurityTrust*` with `eslint-comments/require-description`. Human — whether the comment is adequate and whether an `innerHTML` binding receives user data.

8.7 File uploads

[NEITHER] Client-side size and type checks are UX. The server re-validates by inspecting content, stores privately, and serves through expiring signed URLs.

```
async upload(file: File): Promise<string> {
  if (file.size > 10 * 1024 * 1024) throw new Error('Maximum size is 10 MB');

  // 1. Our API authorises and returns a short-lived presigned PUT URL.
  const { uploadUrl, objectKey } = await firstValueFrom(
    this.http.post<PresignResponse>(`${currentConfig().apiBaseUrl}/uploads/presign`,
      { filename: file.name, contentType: file.type, size: file.size }),
  );
```

```

);

// 2. Browser PUTs straight to S3. No AWS credentials in the browser.
await fetch(uploadUrl, { method: 'PUT', body: file, headers: { 'Content-Type': file.type } });

// 3. Our API verifies the stored object before treating it as valid.
return objectKey;
}

```

The bucket is private. Reads go through a presigned GET with a short expiry, never a public object URL.

8.8 Web3: the chain is the source of truth

[NEITHER] Private keys never touch application code — signing happens in the user's wallet. The backend never trusts a transaction hash, amount or status reported by the browser.

Reason: a client can submit someone else's hash, an unconfirmed one, or a fabricated success. The backend re-fetches from chain and verifies sender, recipient, amount and confirmation depth before any state changes.

```

// The browser's job ends at "I submitted this hash."
const hash = await walletClient.writeContract({
  address: currentConfig().contracts['exchange'],
  abi: exchangeAbi,
  functionName: 'purchase',
  args: [listingId, amountInBaseUnits], // bigint, base units
});

await firstValueFrom(
  this.http.post(`${currentConfig().apiBaseUrl}/orders/${orderId}/submitted`, { hash }),
);
// The UI waits for OUR backend to confirm, not for the wallet.

```

Contract addresses and chain IDs come from `APP_CONFIG`, never inline. A hardcoded mainnet address in a staging build is how test transactions reach production contracts.

9. Dependencies & configuration

- **[NEITHER]** A new dependency needs a line in the PR description: what it does, why we cannot do it in ~50 lines ourselves, last release date, bundle cost. *Reason:* every dependency is permanent attack surface and future upgrade work.
- **[CI]** Lock file always committed; exact versions pinned for direct dependencies. **Check:** `npm ci` fails on a missing or stale lock file; `save-exact=true` in `.npmrc` plus a check for `^/~` on direct dependencies.
- **[CI]** `npm audit` / dependency scan passes per CI policy; High findings block, Medium requires triage. **Check:** `npm audit --audit-level=high` or Trivy as a pipeline step.
- **[NEITHER]** Unmaintained packages — no release in roughly two years — are flagged for replacement rather than adopted.
- **[NEITHER]** Use the stable release, not the newest. The newest version carries the bugs nobody has hit yet, and being first to find them is not a benefit to a production platform.
- **[NEITHER]** Config comes from `APP_CONFIG`, never inline in a service. Removing a feature removes its dependency in the same PR.

10. Maintainability & complexity

- **[LINTER]** `strict: true` in `tsconfig`. Prettier and ESLint are the formatting authority; no formatting debates in review. **Check:** `tsc`, `prettier --check`, ESLint — all three in CI.
- **[LINTER]** Max ~75 lines per method, max 3 levels of nesting, max ~400 lines per component file. **Check:** `max-lines-per-function`, `max-depth`, `max-lines`.
- **[LINTER]** All components use `ChangeDetectionStrategy.OnPush`. *Reason:* default change detection is our most common cause of unexplained slowness. **Check:** `@angular-eslint/prefer-on-push-component-change-detection`.
- **[NEITHER]** Components render; services decide. HTTP, transformation and business rules go in services. **Check:** backstop — `no-restricted-imports` of `HttpClient` in `*.component.ts`.
- **[LINTER + NEITHER]** No function calls in templates, except signal reads. No `.filter()` / `.sort()` chains — precompute in the class or use a

pure pipe. Reason: those re-run on every change detection cycle; signal reads do not and are cheap by design. **Check:** `@angular-eslint/template/no-call-expression` configured to permit signal getters. The rule is worded to exclude signal reads rather than maintained as an `allowList`, because an `allowList` needs updating for every new signal and will drift.

- **[NEITHER]** The third occurrence of duplicated logic gets extracted. The second does not — premature abstraction costs more than duplication.
- **[NEITHER]** Comments explain **why**, never **what**. Public service methods and non-obvious logic carry a short rationale. Every README covers setup, environment variables, and how to run locally and run tests.

11. What we do NOT do

- **[LINTER]** Manual `.subscribe()` without `takeUntilDestroyed()` or the async pipe. Nested subscribes — use `switchMap` / `forkJoin`. **Check:** `rxjs-angular/prefer-takeuntil`, `rxjs-x/no-nested-subscribe`.
- **[LINTER]** `any`. If the shape is unknown, write the interface or use `unknown` and narrow. **Check:** `@typescript-eslint/no-explicit-any`.
- **[NEITHER]** Business logic in a constructor — use `ngOnInit`, or `inject()` plus explicit calls.
- **[LINTER]** `::ng-deep`, `ViewEncapsulation.None`, `!important` to defeat another component's styles. **Check:** `Stylelint selector-pseudo-element-disallowed-list`, `declaration-no-important`; ESLint `no-restricted-syntax` for `ViewEncapsulation.None`.
- **[LINTER]** Direct DOM access — `document.querySelector`, `getElementById`. Use template refs, signals or `Renderer2`. **Check:** `no-restricted-properties`.
- **[LINTER]** `NgModule` for new code. Standalone only. **Check:** `@angular-eslint/prefer-standalone`; existing modules baselined.
- **[LINTER]** `*ngIf` / `*ngFor` in new code. **Check:** `@angular-eslint/template/prefer-control-flow`.
- **[NEITHER]** Mutating `@Input()` values inside a child component. **Check:** signal inputs (`input()`) make this a compile error.
- **[NEITHER]** `setTimeout` to work around a change-detection or ordering problem.
- **[LINTER]** Commented-out code kept “for reference” — git already keeps it. **Check:** `sonarjs/no-commented-code`.
- **[NEITHER]** A shared `UtilService` that becomes a dumping ground. If it has thirty unrelated exports, it is not a utility.
- **[NEITHER]** Silent `catchError(() => of([]))` that hides a real failure as an empty list.
- **[NEITHER]** Mapping `snake_case` to `camelCase` inside a feature. Fix the endpoint (see 1.1).

12. Money and numeric values

[NEITHER] Never use JavaScript `number` arithmetic on a monetary or token amount.

`0.1 + 0.2` is `0.30000000000000004`. On a financial platform that produces defects found by clients, not by us.

Kind	Type in the app	Why
Fiat currency	<code>string</code> in minor units (“1050” = ₹10.50), or a decimal library	<code>number</code> loses precision above 2^{53} and on fractions
Token / on-chain	<code>bigint</code> in base units (wei, etc.)	Exact, and matches what the chain returns

Amounts stay in base units **through the whole application**. Conversion to display units happens only at the presentation boundary, in a pipe.

```
// src/app/shared/pipes/amount.pipe.ts
@Pipe({ name: 'amount', standalone: true, pure: true })
export class AmountPipe implements PipeTransform {
  transform(value: string | bigint, decimals = 2, symbol = '₹'): string {
    const raw = typeof value === 'bigint' ? value : BigInt(value);
    const base = 10n ** BigInt(decimals);
    const whole = raw / base;
    const frac = (raw % base).toString().padStart(decimals, '0');
    return `${symbol}${whole.toLocaleString('en-IN')}.${frac}`;
  }
}
```

```
<span>{{ loan.principalMinor | amount }}</span>
<span>{{ position.amountWei | amount:18:'ETH ' }}</span>
```

Never:

```
const total = parseFloat(a) + parseFloat(b);    // ✕
const fee = amount * 0.025;                  // ✕
<span>{{ loan.principal / 100 }}</span>        // ✕
```

Totals, fees and interest are calculated on the backend. The frontend displays what it is given. If the UI needs to show a computed total before submission, it asks the API rather than reproducing the formula — two implementations of one rule will disagree eventually.

13. Components and the UI

Most of this standard protects us from defects. This section protects us from **rework**.

Our clients change designs during a project and after launch. Whether a redesign costs three days or three weeks is decided when the components are first written, not when the new Figma file arrives. That difference is usually not billable.

13.1 The test

When the design changes, what has to change in the code?

- **Tokens, styles and templates** → the architecture is right.
- **Services, state shapes, types or data fetching** → the separation was wrong, and we are paying for it in rework.

Ask this of your own component before you open a PR. It is a better guide than any individual rule below.

13.2 Tailwind is the styling system

[NEITHER] All new Angular and React projects use Tailwind. Existing projects keep the styling system they have and consume the same tokens (13.3).

Reason, in order of weight on this team:

1. **Tailwind constrains by construction.** There is no class for `13px`. A developer cannot go off-scale without arbitrary-value syntax, which is visible in review. With hand-written CSS, nothing prevents an off-scale value except a linter someone configured and a reviewer who noticed.
2. **There is no cascade to fight.** Specificity wars, `!important` escalation and “why did this change when I edited another file” are the most common CSS defects on a junior team. Tailwind does not have them.
3. **Dead styles cannot accumulate.** Delete a component and its styles go with it. A stylesheet outlives the markup it styled, and nobody dares delete a rule they cannot prove is unused.
4. **One system across two frameworks.** A developer moving between the Angular and React teams does not relearn styling, and there is one thing to review.

Where hand-written CSS is still correct. Tailwind confines CSS; it does not remove it. CSS or SCSS is permitted in exactly four places:

Permitted	Example
The token file	<code>tokens.css</code> — the one place raw values live
The global layer	Resets, font loading, <code>@keyframes</code> , print styles
Third-party overrides	Styling a vendor component we do not control
Existing non-Tailwind projects	Kept as-is, using the same tokens

Anything outside that list is written in Tailwind. “I needed SCSS for this” is answered by the table, not by preference.

A component with more than a handful of utility classes repeated across files is extracted into a component (13.6), not into a stylesheet. Repetition in templates is a signal to extract markup, not to start a parallel CSS system.

13.3 Design tokens are the single source of truth

[LINTER] Colours, spacing, typography, radii, shadows and breakpoints are defined once, as CSS custom properties. A component never contains a raw value.

Reason: a rebrand, a theme, or a white-label build for a second client should touch one file. Hardcoded values turn that into an audit of every component.

Tokens live in plain CSS custom properties, not in `tailwind.config.js`. A JavaScript config is invisible to hand-written CSS, and a colour defined in two places will diverge. One file, consumed by everything.

```
/* src/styles/tokens.css – the only file where a raw value appears */
:root {
  /* ---- Palette: raw brand values, referenced only by the semantic layer ---- */
  --brand-500: #1a6ea8;
  --brand-600: #14557f;
  --neutral-0: #ffffff;
  --neutral-100: #f4f6f8;
  --neutral-600: #5b6670;
  --neutral-900: #13202b;
  --red-500: #c0392b;
  --amber-500: #d29922;
  --green-600: #2ea043;

  /* ---- Semantic: what components actually use ---- */
  --color-surface: var(--neutral-0);
  --color-surface-muted: var(--neutral-100);
  --color-text: var(--neutral-900);
  --color-text-muted: var(--neutral-600);
  --color-border: #d8dee4;
  --color-primary: var(--brand-500);
  --color-primary-hover: var(--brand-600);
  --color-danger: var(--red-500);
  --color-warning: var(--amber-500);
  --color-success: var(--green-600);

  /* ---- Spacing scale. Nothing between these steps. ---- */
  --space-1: 0.25rem; --space-2: 0.5rem; --space-3: 0.75rem;
  --space-4: 1rem; --space-6: 1.5rem; --space-8: 2rem;
  --space-12: 3rem; --space-16: 4rem;

  /* ---- Type scale ---- */
  --text-xs: 0.75rem; --text-sm: 0.875rem; --text-base: 1rem;
  --text-lg: 1.125rem; --text-xl: 1.5rem; --text-2xl: 2rem;
  --font-sans: 'Inter', system-ui, sans-serif;

  /* ---- Radii and elevation ---- */
  --radius-sm: 4px; --radius-md: 8px; --radius-lg: 16px; --radius-full: 9999px;
  --shadow-1: 0 1px 2px rgb(0 0 0 / 0.06);
  --shadow-2: 0 4px 12px rgb(0 0 0 / 0.10);
}

/* A theme overrides the semantic layer only – never the palette, never a component. */
[data-theme='dark'] {
  --color-surface: var(--neutral-900);
  --color-surface-muted: #1c2b38;
  --color-text: var(--neutral-0);
  --color-text-muted: #9aa7b2;
  --color-border: #2c3a46;
}
```

Two layers, and the distinction matters. The palette holds raw values. The semantic layer names a use. Components reference only the semantic layer, which is why a theme or a client rebrand is a handful of lines rather than a search-and-replace.

Tailwind consumes the tokens

Tailwind v4, CSS-first:

```
/* src/styles/theme.css */
@import 'tailwindcss';
@import './tokens.css';

@theme inline {
  --color-surface: var(--color-surface);
  --color-primary: var(--color-primary);
  --color-danger: var(--color-danger);
  --spacing-1: var(--space-1);
  --spacing-4: var(--space-4);
  --radius-md: var(--radius-md);
  --font-sans: var(--font-sans);
}
```

On Tailwind v3, the JS config points at the same variables:

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        surface: 'var(--color-surface)',
        primary: 'var(--color-primary)',
        danger: 'var(--color-danger)',
      },
      spacing: { 1: 'var(--space-1)', 4: 'var(--space-4)' },
      borderRadius: { md: 'var(--radius-md)' },
    },
  },
};
```

The CSS that remains consumes the same tokens

```
// Never redeclare a value. Reference the custom property.
.card {
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-md);
  padding: var(--space-4);
  box-shadow: var(--shadow-1);
}
```

Never:

```
$primary: #1a6ea8;           // ✗ a second source of truth
.card { padding: 13px; }     // ✗ off-scale
.btn { color: #fff; }       // ✗ raw value, and breaks in dark theme
```

Check: `stylelint-declaration-strict-value` on `color`, `background-color`, `border-color`, `fill`, `stroke`, `padding`, `margin`, `font-size`, `border-radius` and `box-shadow`, permitting only `var(...)`, `inherit`, `currentColor` and `transparent`.

13.4 Figma and code use the same component names

[NEITHER] A component's name in Figma and its name in the codebase are the same name.

They are not today. When a designer says “update the Card” nobody can map that to a file without opening several, and a design handoff becomes a translation exercise that loses detail every time.

Fixing it is one session between the lead and the designer, per project:

- Pick the name the design uses, unless it is clearly wrong. The designer renaming is usually cheaper than the developer renaming.
- `Card/Default`, `Card/Compact` in Figma → `app-card` with a `variant` input in code.
- A Figma variant becomes an `@Input`, not a second component.
- Record the mapping in the project README where a one-to-one name is impossible.

New projects start aligned. Existing projects align at the next design revision for the components that revision touches — not as a separate renaming exercise.

13.5 Three kinds of component

[NEITHER] Every component is one of these, and the kind decides what it may contain.

Kind	Lives in	May inject	May do
Page	features/<x>/pages/	Services, router	Fetch data, own view state, compose children
Feature	features/<x>/components/	Nothing	Know the domain; typed inputs and outputs only
Presentational	shared/components/	Nothing	Render what it is given. Knows no domain at all

A **presentational component with an injected service is the single most expensive mistake in this section**. It cannot be reused, cannot be tested in isolation, and cannot survive a redesign that moves it to a different screen.

```
// ✓ Presentational. Reusable, testable, survives any redesign.
@Component({
  selector: 'app-status-pill',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<span class="pill" [class]="tone()">{{ label() }}</span>`,
})
export class StatusPillComponent {
  readonly label = input.required<string>();
  readonly tone = input<'neutral' | 'success' | 'danger'>('neutral');
}
```

```
// ✗ Not presentational. It fetches, so it can only ever be used on one screen.
export class StatusPillComponent {
  private readonly loans = inject(LoanService); // ✗
}
```

Check: backstop — `no-restricted-imports` in `shared/components/**` banning `HttpClient`, `Router`, and `features/*`.

13.6 Inputs and outputs are the contract; markup is not

[NEITHER] The internal markup and styles of a component may change at any time without notice. **Its inputs and outputs may not.**

Changing or removing an input on a shared component is a breaking change: add the new input, migrate the consumers, then remove the old one. Do not edit it in place and fix the compile errors — that is the same work with no migration path and no review of what you changed.

Presentational components take **view data, not domain objects**. A component typed to `Loan` breaks when the API changes; a component typed to `{ title, subtitle, tone }` does not.

13.7 A component never sets its own outer margin

[NEITHER] A component owns everything inside its box. The **parent** owns where it sits and how much space is around it.

Reason: a component carrying `margin-bottom: 16px` fights every reuse, and a layout redesign means editing every child rather than one parent.

```
/* ✗ inside status-pill.component.scss */
:host { margin-bottom: var(--space-4); }

/* ✓ in the parent that places it */
.loan-row { display: flex; gap: var(--space-4); }
```

Use `gap` on the parent. It is the whole answer to spacing between siblings.

13.8 Styles never reach outward

[NEITHER] A component styles itself and its own template. It never styles another component's internals.

`::ng-deep`, `ViewEncapsulation.None` and `!important` are already banned in section 11. The positive rule: if a parent needs a child to look different, the child exposes an input or a documented CSS custom property.

```
// The child decides what is themeable.
@Component({
  selector: 'app-card',
  styles: [
    :host {
      background: var(--card-surface, var(--color-surface));
      padding: var(--card-padding, var(--space-4));
    }
  ],
})
```

```
/* The parent sets the variable. No reaching inside. */
.compact-list app-card { --card-padding: var(--space-2); }
```

13.9 Responsive and accessible as it is written

[NEITHER] Mobile-first, breakpoints from tokens, accessibility built in at the time the component is written.

Retrofitting accessibility after a redesign costs several times what building it in costs, and for clients in regulated or public-sector work it is a contractual exposure rather than a quality nicety. If a client contract carries an accessibility obligation, the lead confirms which standard applies before the first screen is built.

Minimum for every component:

- Semantic elements — `<button>` for actions, `<a>` for navigation. Never a `<div>` with a click handler.
- Every input has a `<label>`; every icon-only control has an `aria-label`.
- Focus is visible and never removed with `outline: none`.
- Interactive targets at least 44×44px on touch.
- Text contrast meets 4.5:1 — checked against the tokens once, not per screen.

Check: `@angular-eslint/template/accessibility-*` rules catch the mechanical half. The rest is checklist item 14.

13.10 Starting the shared component library

We do not have one today. The way to get one is not to sit down and design it.

[NEITHER] **A component graduates to shared/ on its third use, not its first.** Two uses is a coincidence; three is a pattern, and by then you know which parts actually vary.

When it graduates it gains:

- A named owner — the lead, unless someone else is recorded in the README
- A documented input/output contract
- A test, if it holds any logic
- The breaking-change rule in 13.5

Keep it as a `shared/` folder inside one repo. **Do not extract it into an npm package until two repositories genuinely need the same component** — a published package adds versioning, release and upgrade work, and we have one Angular lead.

13.11 Visual regression — next, not now

[NEITHER] A token change can alter thirty screens silently. A screenshot baseline catches that; nothing else does.

This needs tooling we do not have, so it is a planned improvement rather than a day-one rule. Until it exists, a PR that changes `tokens.css` names in its description which screens were checked by hand.

Appendix — ESLint starting point

Rules this standard depends on. Baseline existing violations rather than fixing them all at once: suppress what exists today and fail the build only on new violations.

```
// eslint.config.js (flat config)
export default [
  {
    files: ['src/**/*.ts'],
    rules: {
      '@typescript-eslint/no-explicit-any': 'error',
      '@typescript-eslint/no-non-null-assertion': 'error',
      '@typescript-eslint/no-unused-vars': 'error',
      '@typescript-eslint/no-magic-numbers': ['warn', { ignore: [0, 1, -1] }],
      'no-console': 'error',
      'max-lines-per-function': ['error', 75],
      'max-depth': ['error', 3],
      'max-lines': ['error', 400],
      'no-empty': 'error',
      'sonarjs/no-commented-code': 'error',
      'rxjs-x/no-nested-subscribe': 'error',
      'rxjs-angular/prefer-takeuntil': ['error', { alias: ['takeUntilDestroyed'] }],
      '@angular-eslint/prefer-standalone': 'error',
      '@angular-eslint/prefer-on-push-component-change-detection': 'error',
      'no-restricted-properties': ['error',
        { object: 'localStorage', property: 'setItem' },
        { object: 'sessionStorage', property: 'setItem' },
        { object: 'document', property: 'querySelector' },
        { object: 'document', property: 'getElementById' },
      ],
      'no-restricted-imports': ['error', {
        paths: [
          { name: '@angular/forms', importNames: ['UntypedFormGroup', 'UntypedFormControl', 'UntypedFormBuilder'],
            message: 'Use typed reactive forms. See standard 3.' },
        ],
      }],
      'no-restricted-syntax': ['error', {
        selector: 'CallExpression[callee.property.name=/^bypassSecurityTrust/]',
        message: 'Requires an APPROVED BYPASS comment and a second reviewer. Standard 8.6.',
      }],
    },
  },
  {
    files: ['src/app/core/logging/**/*.ts'],
    rules: { 'no-console': 'off' },
  },
];
```

Also required: `eslint-plugin-check-file` for file naming, `eslint-plugin-boundaries` for cross-feature imports, `@angular-eslint/template/prefer-control-flow` and `no-call-expression` for templates, and `@angular-eslint/template/accessibility-*` for section 13.9.

Add to `shared/components/**` so presentational components stay presentational (13.5):

```
{
  files: ['src/app/shared/components/**/*.ts'],
  rules: {
    'no-restricted-imports': ['error', {
      paths: [
        { name: '@angular/common/http', message: 'Presentational components do not fetch. Standard 13.5.' },
        { name: '@angular/router', message: 'Presentational components do not navigate. Standard 13.5.' },
      ],
      patterns: [{ group: ['**/features/*'], message: 'Shared components know no domain. Standard 13.5.' }],
    }],
  },
}
```

Appendix — Stylelint starting point

Enforces the token rules in section 13.3. Works for both Tailwind and SCSS projects.

```
// .stylelintrc.js
module.exports = {
  extends: ['stylelint-config-standard-scss'],
  plugins: ['stylelint-declaration-strict-value'],
  rules: {
    'scale-unlimited/declaration-strict-value': [
      ['/color$/', 'fill', 'stroke', 'padding', 'margin', 'gap',
        'font-size', 'border-radius', 'box-shadow'],
      {
        ignoreValues: ['inherit', 'currentColor', 'transparent', 'none', 'auto', '0'],
      }
    ],
  },
}
```

```
    ignoreKeywords: { '/color$/' : ['inherit', 'currentColor', 'transparent'] },
    disableFix: true,
    message: 'Use a token from tokens.css. See standard 13.3.',
  },
],
'declaration-no-important': true,
'selector-pseudo-element-disallowed-list': ['ng-deep'],
},
ignoreFiles: ['src/styles/tokens.css'],
};
```

`tokens.css` is the one file exempt from the rule, because it is where raw values are allowed to exist.